System Architecture Evaluation: Autonomous Edge AI Video Pipeline on WSL2

1. Architectural Overview and Edge Computing Constraints

The engineering and deployment of a fully autonomous local AI video editing pipeline on consumer-grade or mobile workstation hardware presents a multifaceted orchestration challenge. The target infrastructure is defined by an Intel Core Ultra 7 255HX processor paired with an NVIDIA RTX 5080 Mobile Graphics Processing Unit (GPU). This environment is strictly bound by a 16GB Video RAM (VRAM) limitation and 32GB of unified system RAM. Furthermore, the reliance on a Windows 11 host operating system utilizing the Windows Subsystem for Linux (WSL2) with an Ubuntu 24.04 userland, executed via a custom PyTorch Nightly Docker container, introduces deep virtualization layers. These paravirtualized layers fundamentally alter how hardware resources are addressed, managed, pinned, and released across the host-to-guest boundary.

The proposed architecture relies on a "Hub-and-Spoke" VRAM management paradigm. This design aims to mitigate the strict 16GB VRAM constraint by deploying a central CPU orchestrator (the Hub) to manage systemic state, while sequentially loading, executing, and terminating ephemeral "Spoke" models, which include Automatic Speech Recognition (ASR), YOLO for spatial object detection, and the Qwen Large Language Model for advanced semantic reasoning. By ensuring that these highly parameterized models do not reside in memory concurrently, the system targets a peak VRAM utilization of less than 14GB, leaving a necessary 2GB buffer for operating system display drivers and the WSL2 translation layer.

To prevent the catastrophic saturation of the Peripheral Component Interconnect Express (PCIe) bus, the architecture establishes a zero-copy video processing pipeline. This is achieved utilizing FFmpeg 7.1, compiled with explicit hardware acceleration flags (--enable-cuda, --enable-nvdec, --enable-nvenc), interfacing directly with PyNvVideoCodec and Kornia. Frames are intended to be pinned in the GPU memory space and passed as raw pointers to the respective Spoke models. A background "Locker" process is deployed to hold a 100MB Shared CUDA Buffer via Inter-Process Communication (IPC) handles, ensuring that the sequential termination of Spoke processes does not destroy the underlying video tensors. Finally, the system dictates that video is processed in discrete 60-second chunks, utilizing FFmpeg's fast-seek -ss parameter for zero-copy stream stream cutting, targeting a 50-millisecond perceptual offset.

While this architectural framework is theoretically robust, mapping it to the operational realities

of the WSL2 kernel bridge, the PyTorch CUDA Caching Allocator, and the physical constraints of Blackwell-generation hardware encoders exposes several critical vulnerability points. Rapid sequential process initialization and termination within a containerized Linux environment carry extremely high risks of POSIX shared memory (/dev/shm) exhaustion, memory handle leakage, and catastrophic VRAM fragmentation. Furthermore, passing CUDA IPC handles between ephemeral processes introduces severe risks of invalid device context access when the originating process is destroyed. Finally, the assumption that a 50-millisecond perceptual offset can be maintained during a stream-copy operation fundamentally contradicts the physical mechanics of inter-frame video compression formats.

The following exhaustive analysis critically evaluates these architectural bottlenecks, assesses the viability of modern alternatives such as the NVIDIA Triton Inference Server, and prescribes concrete, low-level engineering solutions to ensure pipeline stability, zero-copy pointer integrity, and millisecond-accurate frame extraction on the RTX 5080.

2. WSL2 and Docker IPC Memory Management Dynamics

The execution of a rapid sequence of PyTorch Spoke processes inside a Docker container running on a WSL2 backend creates a highly volatile system memory environment. When ephemeral models such as YOLO or Qwen are initialized, execute their forward passes, and subsequently terminate to free their VRAM footprint, the underlying Ubuntu operating system and the PyTorch multiprocessing modules must continuously allocate, map, populate, and unmap shared memory segments.

2.1 The /dev/shm Fragmentation and Allocation Bottleneck

Within standard Linux environments, POSIX shared memory, mounted as a temporary file system at /dev/shm, is heavily utilized by PyTorch's multiprocessing library. This is particularly relevant for dataloader workers and tensor sharing across process boundaries.[1] By default, the Docker daemon allocates a severely restricted 64MB to the /dev/shm partition.[1] When multiple PyTorch processes spawn sequentially and attempt to push high-resolution video frames or large embedding matrices into shared memory queues, this 64MB limit is breached almost instantaneously. When the shared memory segment is exhausted, the container will suffer a fatal Bus error (core dumped) or yield a RuntimeError: unable to write to file </torch_...>: No space left on device.[2]

However, the architectural challenge extends far beyond merely expanding the initial allocation limits. The specific requirement of rapid process death—a core component of the Hub-and-Spoke design—introduces a severe risk of memory leakage and page fragmentation. Under WSL2, the lightweight utility virtual machine (VM) dynamically allocates system RAM, which is represented in the Windows host task manager as the vmmem process. When Docker containers rapidly allocate and deallocate memory, the WSL2 kernel often fails to return freed

pages to the Windows host promptly. This results in ballooning vmmem usage that eventually consumes all 32GB of available system RAM, triggering the Linux Out-Of-Memory (OOM) killer and permanently halting the orchestration pipeline.[6]

2.2 PyTorch IPC Reference Counting Anomalies

A heavily documented anomaly exists within PyTorch's CUDA IPC reference counting mechanisms that directly threatens the stability of the ephemeral Spoke architecture. When a tensor is shared across processes, PyTorch relies on a reference counter to dictate when the underlying CUDA memory can be safely released back to the allocator. In multi-consumer IPC scenarios involving rapid process termination, the integer subtraction operation (ref -= 1) is performed natively as an int64 type, but the output is frequently cast as a uint64 type by the underlying C++ engine.[9]

If a Spoke process terminates irregularly, is forcefully killed by the orchestrator, or drops a reference during an asynchronous transfer, this type conversion triggers an integer underflow. The result is an astronomically large reference count assigned to the tensor.[9] When this underflow occurs, the PyTorch Caching Allocator permanently locks the associated VRAM and system memory, falsely believing that millions of processes are currently accessing the buffer. This results in an unrecoverable, silent memory leak that persists across process boundaries, eventually starving the system of VRAM and requiring a complete destruction and restart of the Docker container to clear.[9]

2.3 Prescribed Docker and WSL2 System Configurations

To stabilize the Hub-and-Spoke architecture, prevent host-system RAM exhaustion, and override the default Docker limitations, multi-layered constraints must be implemented at both the WSL2 kernel level and the Docker runtime configuration.

Configuration Layer	Required Parameter	Engineering Rationale
WSL2 Kernel Limit	.wslconfig implementation: [wsl2] memory=24GB processors=12 swap=8GB	Hard-limits the vmmem process from consuming the entire 32GB of host RAM. Leaving an 8GB reserved buffer for the Windows OS prevents system-wide lockups during aggressive PyTorch memory reclamation.[6]

Docker IPC Namespace	--ipc=host	Bypasses the default 64MB /dev/shm namespace limit entirely by mapping the container's IPC namespace directly to the host WSL2 OS. This grants PyTorch access to the full 24GB virtual allocation.[2]
Docker SHM Sizing	--shm-size=16gb	An alternative to --ipc=host that explicitly defines a massive shared memory pool exclusively for the container. This is critical for maintaining the 100MB shared CUDA buffer securely across process boundaries.[2]
Docker Resource Limits	--ulimit memlock=-1	Essential for preventing page-outs of pinned memory blocks. This ensures that the zero-copy video frames held in system RAM are not swapped to the virtual disk, which would instantly destroy the latency benefits of the pipeline.[14]

3. PyTorch CUDA Allocator Configurations for Strict VRAM Limitations

Maintaining the strict 16GB VRAM constraint on the RTX 5080 Mobile while continuously swapping out highly parameterized Spoke models (ASR, YOLO, Qwen) presents a severe fragmentation risk. The default behavior of PyTorch's memory management is highly unsuited for rapid model churn.

3.1 The Mechanics of VRAM Fragmentation

To avoid the extreme latency penalties associated with calling the underlying CUDA runtime API (cudaMalloc and cudaFree) for every mathematical operation, PyTorch utilizes a proprietary Caching Allocator. This allocator requests massive blocks of memory from the GPU and retains

them in a private cache pool even after the variables are deleted in Python.[15] The goal is to reach a steady state where the program runs without needing to request new memory from the OS.[15]

However, when an ASR model loads, executes, and completely terminates, the PyTorch allocator intercepts the memory release. Instead of returning the 4GB of VRAM to the physical GPU, it retains it in its internal pool as "reserved but unallocated" memory. When the subsequent Qwen model (an LLM) spins up and requests memory for its massive transformer weights and Key-Value (KV) cache, the allocator attempts to fulfill it from this reserved pool. Because the tensor shapes, block alignments, and parameter sizes of the Qwen model differ drastically from the ASR model, the existing memory blocks cannot be reused efficiently. The allocator is forced to split blocks into smaller segments.

Over a few cycles, this results in severe VRAM fragmentation. The total free memory reported by nvidia-smi might appear as 8GB, but the largest contiguous block available might only be 50MB. When Qwen requests a 1GB contiguous block for a matrix multiplication, the allocator fails, triggering a CUDA out of memory (OOM) error despite sufficient mathematical availability.[16]

3.2 Evaluating backend:cudaMallocAsync

The environment variable PYTORCH_CUDA_ALLOC_CONF governs the behavior of the caching allocator. One potential configuration is backend:cudaMallocAsync, which completely disables PyTorch's native implementation and forces the system to use CUDA's built-in asynchronous allocator (introduced in CUDA 11.4).[19]

The theoretical advantage of cudaMallocAsync is that it delegates defragmentation directly to the NVIDIA display driver. If a memory allocation request cannot be serviced due to fragmentation, the CUDA driver can dynamically defragment the pool by remapping unused memory to a contiguous portion of the GPU's virtual address space.[20] While this sounds ideal for the Spoke architecture, empirical data indicates that relying on the asynchronous backend occasionally introduces severe overhead, strips the PyTorch profiler of meaningful memory statistics, and has been documented to leave massive amounts of memory "missing" during heavy iteration cycles.[18]

3.3 The Optimal Solution: expandable_segments:True

To mitigate fragmentation within the 16GB limit robustly, the PyTorch native allocator must be instructed to utilize CUDA Virtual Memory Management (VMM) APIs. This is achieved by setting expandable_segments:True.[16]

Normally, for large allocations (greater than 2MB), the allocator calls cudaMalloc to get blocks that perfectly match the user request. With expandable segments enabled, the allocator creates CUDA allocations that can be dynamically expanded later.[19] It effectively allows the

allocator to map discontiguous physical memory pages on the GPU into perfectly contiguous virtual memory addresses for PyTorch to use. When the YOLO model unloads and the Qwen model loads, the allocator can seamlessly stitch together the fragmented remnants of YOLO's footprint to form the massive contiguous blocks Qwen requires. This is highly effective at preventing artificial OOM errors in high-churn model loading environments.[16]

3.4 Tuning the Maximum Split Size

To further defend the 16GB limitation, the allocator must be prevented from pulverizing its memory blocks. The max_split_size_mb parameter restricts the allocator from splitting blocks larger than a defined threshold.[18] If a 1GB block is free, and a process requests 10MB, a naive allocator will split the 1GB block, leaving a 990MB block. Over time, this creates thousands of tiny inactive blocks. By setting a split limit, performance cost can range from zero to substantial depending on the pattern, but it acts as a failsafe against terminal fragmentation.[24]

Allocator Configuration	Implementation Syntax	Engineering Rationale
Virtual Memory Expansion	expandable_segments:True	Instructs the native allocator to use CUDA VMM APIs to map discontiguous physical pages into contiguous virtual addresses, solving cross-model fragmentation.[16]
Split Size Limitation	max_split_size_mb:512	Prevents the allocator from fracturing large memory blocks (over 512MB) to satisfy minor tensor requests, preserving large contiguous spaces for the initialization of subsequent Spoke models.[18]

The optimal environment variable injection for the Docker container runtime is therefore:

```
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True,max_split_size_mb:512
```

4. CUDA IPC Handle Volatility in Hub-and-Spoke Topologies

The system architecture utilizes a background "Locker" process to hold a 100MB Shared CUDA Buffer via IPC Handles. This design theoretically allows sequentially loaded Spoke processes to access the memory without triggering PCIe bus transfers, copying data directly from device to device. This methodology leverages the underlying cudaIpcGetMemHandle and cudaIpcOpenMemHandle APIs to share device pointers across process boundaries.[26]

4.1 Lifecycle and Volatility of Ephemeral CUDA Pointers

Inter-Process Communication (IPC) within the CUDA architecture operates under strict ownership and lifecycle paradigms. Any device memory pointer or event handle created by a host thread can be directly referenced by any other thread within the same process. However, to access device memory across different processes, an application must utilize CUDA IPC or VMM APIs to create process-portable handles.[26]

When a process allocates memory on the GPU (e.g., via cudaMalloc), it establishes a CUDA context and assumes total ownership of that physical memory space. If that process utilizes cudaIpcGetMemHandle to create an IPC handle and passes it via a socket or queue to a secondary process, the secondary process can open the handle and interact with the memory safely.[26]

However, a fundamental limitation dictates the volatility of these handles: if the originating process (the owner) terminates, the NVIDIA display driver immediately initiates a rigorous cleanup routine. It destroys the underlying device context and forcefully frees the physical memory to prevent leaks. Consequently, any IPC handles passed to other processes instantly become dangling pointers. If a surviving process attempts to read or write to that handle, the driver will raise a fatal CUDA error: an illegal memory access was encountered or an invalid device context exception, immediately and violently crashing the application.[28]

In a Hub-and-Spoke architecture where Spoke processes are strictly ephemeral—designed to execute and completely terminate to clear their VRAM footprint before the next Spoke spins up—they cannot act as the originators of the shared memory. If the ASR Spoke allocates the zero-copy buffer, passes it to the Qwen Spoke, and exits, the pipeline will suffer a catastrophic fault.

4.2 The Persistent "Locker" Process Pattern

Given the extreme volatility of ephemeral pointers, the deployment of a persistent background "Locker" process is not merely an optimization; it is a strict architectural necessity to maintain the zero-copy pipeline across process deaths.

The Locker process must execute first, initialize its own CUDA context, allocate the 100MB buffer, and generate the IPC handles. Crucially, the Locker process must then enter an infinite wait state, ensuring its process ID (PID) remains active for the entire duration of the video processing pipeline. The ephemeral Spoke models spin up, import the IPC handle from the

Locker using PyTorch's multiprocessing abstractions, read the video frames, and terminate.[32] Because the Locker process remains alive, the underlying memory context remains valid across the sequential deaths of the Spokes.

To safely pass these tensors in Python, the system must utilize PyTorch's multiprocessing queues (multiprocessing.Queue or multiprocessing.SimpleQueue). When a PyTorch CUDA tensor is placed in a queue, the framework automatically generates the underlying IPC handles under the hood.[32] It is absolutely imperative that the Locker process retains a hard reference to the original tensor variable in its local scope; otherwise, Python's automatic garbage collector will issue a __del__ call when the variable goes out of scope, triggering a cudaFree operation and violating the memory persistence, causing the same illegal access faults.[32]

Furthermore, the implementation must account for synchronization. The Locker and the Spokes are operating asynchronously. The architecture must utilize host-based IPC to pass CUDA event handles alongside the memory handles. This allows the Spoke process to execute cudaStreamWaitEvent, ensuring that the GPU kernel does not attempt to read the 100MB buffer before the previous process has finished writing to it.[27]

4.3 Evaluation of NVIDIA Triton Shared Memory Alternatives

A critical evaluation was conducted to determine if modern, industry-standard ML infrastructure—specifically the NVIDIA Triton Inference Server—could replace the custom Python Locker process to manage the shared memory pooling more elegantly. Triton provides advanced CUDA shared-memory extensions that allow clients to communicate input and output tensors directly via GPU VRAM. This bypasses the overhead of gRPC or REST network payloads and allows for pristine zero-copy inference across multiple models.[33]

Under normal bare-metal Linux deployments, Triton represents a far superior, enterprise-grade alternative to maintaining a custom Python-based Locker process. However, the operational environment is WSL2 (Ubuntu 24.04). Deep analysis of Triton Inference Server deployments within Windows/WSL2 environments reveals a critical compatibility failure.

When attempting to initialize CUDA shared memory pools inside Triton under WSL2—particularly when utilizing Business Logic Scripting (BLS) to pass tensors between models with pb_utils.PreferredMemory(pb_utils.TRITONSERVER_MEMORY_GPU)—the server invariably throws a fatal error: Failed to initialize CUDA shared memory pool in Python stub: Failed to open the cudaIpcHandle. error: invalid resource handle.[34]

Due to the complex nature of WSL2's virtualization layer, where the Linux kernel interacts with the Windows Display Driver Model (WDDM) via a paravirtualized interface, Triton's specific implementation of CUDA IPC handle registration fails to map correctly across the boundary.[34] When this handle registration fails, Triton is designed to maintain uptime by automatically falling back to TRITONSERVER_MEMORY_CPU. This fallback violently defeats the entire purpose of the zero-copy pipeline, forcing the system to download the 100MB buffers from VRAM to system

RAM over the PCIe bus, and re-uploading them for every single Spoke execution, resulting in massive latency bottlenecks.[34]

Consequently, utilizing NVIDIA Triton Shared Memory is definitively counter-indicated for this specific WSL2 architecture. The custom Python "Locker" process, relying on standard torch.multiprocessing IPC handles over --ipc=host container boundaries, remains the most robust and functional method available.

5. Zero-Copy Pipeline Integrity and Pinned Memory Bottlenecks

The pipeline utilizes FFmpeg 7.1, PyNvVideoCodec, and Kornia to decode the 60-second video chunks directly into GPU memory. PyNvVideoCodec is NVIDIA's successor to the Video Processing Framework (VPF), providing Python bindings directly over the optimized C++ APIs of the Video Codec SDK.[35]

5.1 PyNvVideoCodec and DLPack Integration

To ensure the frames decoded by PyNvVideoCodec are seamlessly transferred into PyTorch tensors without invoking disastrous CPU memory roundtrips, the system must leverage DLPack. DLPack provides a standardized, open-source in-memory tensor structure designed specifically for zero-copy data exchange between distinct computational frameworks.[36] PyNvVideoCodec natively implements the Python DLPack protocol via the __dlpack__() and __dlpack_device__() magic methods on its decoded frame objects.[36]

To engineer this correctly, the pipeline must strictly adhere to the following initialization sequence:

1. The PyNvVideoCodec decoder must be initialized with the flag use_device_memory=True. Failing to set this will cause the decoder to silently dump frames into host RAM.[36]
2. The output color format must be explicitly configured to OutputColorType.RGBP (Planar CHW format). Convolutional neural networks in PyTorch expect channel-first memory layouts. If this is not set, PyTorch will be forced to execute an expensive, synchronous contiguous memory copy to reorder the interleaved channels.[36]
3. The decoded frame is directly ingested via the torch.from_dlpack(frame) tensor creation method, which assumes immediate, zero-copy pointer access to the underlying CUDA memory.[36] Furthermore, stream synchronization (torch.cuda.current_stream().synchronize()) must be enforced before passing the pointer to the Locker process.[36]

5.2 The 500MB WSL2 Pinned Memory Limitation

While the architecture currently mandates a 100MB shared CUDA buffer, future expansions of

the pipeline may attempt to increase this buffer size or utilize massive amounts of page-locked (pinned) host memory to speed up transfers. Pinned memory is crucial because it allows the GPU to utilize Direct Memory Access (DMA) to fetch data from system RAM without CPU intervention.

However, a critical engineering warning must be issued for the WSL2 environment. The NVIDIA Container Toolkit operating on WSL2 imposes an undocumented hard limit on the allocation of pinned host memory. Extensive testing and issue tracking reveal that attempts to allocate contiguous pinned memory blocks exceeding 500MB consistently fail on WSL2 environments, returning silent errors or crashing the application, regardless of the 32GB of physical RAM available on the host.[14]

This limitation is not present in native Windows or native Linux, but is a unique artifact of the containerized CUDA driver interacting with the WSL2 backend.[14] Therefore, while the 100MB buffer is safe, any architectural adjustments that attempt to pin massive video segments into host RAM must chunk the allocations into sizes strictly below the 500MB threshold to prevent silent pipeline halts.

6. FFmpeg NVENC Frame Snapping and Blackwell Optimizations

The orchestration layer dictates that the source video is processed in 60-second chunks, and the final autonomous clipping relies on the FFmpeg -ss seek command utilizing a 50-millisecond perceptual offset combined with stream copying (-c copy). This methodology requires rigorous critical evaluation, as it violates the core physical mechanisms of video compression formats and will fail to deliver the required accuracy.

6.1 The GOP Structure and the Failure of -c copy Seeking

Modern highly compressed video codecs (such as H.264, HEVC, and AV1) do not store video as a sequence of independent, fully rendered bitmap images. Instead, to achieve maximum spatial and temporal compression, they utilize a Group of Pictures (GOP) structure consisting of three distinct frame types:

- **I-frames (Intra-coded):** Complete, standalone images that require no external data to decode. They function as the anchor points of the video and take up the most data.[37]
- **P-frames (Predictive):** Contain only the mathematical differences (motion vectors and residual data) from previous I-frames or P-frames.[37]
- **B-frames (Bi-directional):** Reference data from both preceding and succeeding frames to achieve maximum compression efficiency.[38]

When utilizing the FFmpeg stream copy parameter (-c copy), the engine is instructed to demux the bitstream and remux it into a new container completely untouched, without passing the

data through a decoder.[39] If a command specifies a cut at exactly $T = 12.050$ seconds (accounting for the 50ms offset), the likelihood of that exact timestamp corresponding perfectly to an I-frame is statistically minuscule.

If the 12.050-second mark falls on a P-frame or a B-frame, it is a physical impossibility for FFmpeg to initiate the copied stream at that exact millisecond. A P-frame consists solely of difference data; without the preceding I-frame to reference, the image cannot be mathematically reconstructed.[37] To resolve this and prevent generating a corrupt, unplayable file, FFmpeg's -c copy logic will automatically override the requested timestamp and snap the cut point backward to the nearest preceding keyframe (I-frame).[37] Depending on the GOP length of the source material, this can snap the cut backwards by several seconds, completely obliterating the 50-millisecond perceptual offset accuracy required by the autonomous pipeline.

6.2 The Reality of "Smart Cutting"

The concept of "Smart Cutting"—where the vast majority of the video is stream-copied losslessly, except for the boundary edges which are decoded and re-encoded to the nearest keyframe—is widely discussed but practically unachievable in pure, unmodified FFmpeg via a single command string. True smart cutting requires highly complex multi-pass scripting to demux the streams, mathematically split the GOP logic, re-encode boundary segments with precisely matched quantization parameters to prevent visual jarring, and concatenate the disparate files via the concat protocol.[40]

For an autonomous, error-resilient Edge AI pipeline, attempting to script a smart cut introduces catastrophic points of failure related to Decoding Time Stamp (DTS) synchronization, audio desync, and container matching. Therefore, to guarantee millisecond-accurate cuts with zero perceptual drift, the specified 60-second chunk *must* be subjected to a full decode and re-encode pass.[38] Given that the segment is only 60 seconds long, the processing overhead on a Blackwell-class GPU is negligible.

6.3 Blackwell NVENC Capabilities and Driver-Level Anomalies

The NVIDIA RTX 5080 Mobile is built upon the Ada Lovelace successor architecture (Blackwell), featuring the 9th generation NVENC application-specific integrated circuits (ASICs).[43] The Blackwell NVENC offers profound advancements, notably a dual-encoder configuration capable of massive parallelization, and a 5% baseline compression efficiency improvement in the AV1 codec over the previous 40-series generation.[44]

To maximize visual quality and strictly match the source material during the re-encode, the optimal codec selection is AV1 via the av1_nvenc plugin.[47] AV1 yields roughly 40% bit rate savings over H.264 at identical Peak Signal-to-Noise Ratio (PSNR) and Video Multimethod Assessment Fusion (VMAF) scores, meaning the re-encoded cut will suffer virtually zero

generational loss.[47]

However, a highly specific silicon and driver-level bug exists within the current Blackwell architecture that must be circumvented. NVIDIA has documented that utilizing the Ultra High Quality tuning flag (-tune uhq) in combination with 10-bit color depth (-highbitdepth 1) on RTX 50-series GPUs (specifically observed across the 5070 Ti, 5080, and 5090 SKUs) causes catastrophic macroblocking artifacts and visual corruption during encoding.[50] While this is slated to be patched in future proprietary driver branches [50], any autonomous pipeline deployed immediately must avoid combining these two parameters to ensure output integrity.

6.4 Engineering the Millisecond-Accurate Re-encode Pipeline

To guarantee exact frame parsing on the RTX 5080 utilizing full hardware acceleration (bypassing the host CPU entirely to preserve resources for the Hub orchestrator), the FFmpeg command must place the seek parameter (-ss) as an input option to utilize fast keyframe seeking, while defining duration (-t) precisely.

The encode parameters must abandon arbitrary bitrates and utilize Constant Quality Variable Bitrate (-rc vbr combined with -cq), utilizing a high-quality preset without triggering the Blackwell UHQ anomaly.

The exact synthesized FFmpeg command for hardware-accelerated, millisecond-accurate cutting is defined as follows:

Bash

```
ffmpeg -hwaccel cuda -hwaccel_output_format cuda -ss 00:01:12.050 -i input.mp4 -t 00:01:00.000 -c:v av1_nvenc -preset p7 -tune hq -rc vbr -cq 19 -b:v 0 -bf 3 -c:a copy -y output_cut.mp4
```

6.5 Architectural Breakdown of the FFmpeg Command

The parameters detailed above are strictly tuned for the RTX 5080 environment:

FFmpeg Parameter	Functional Engineering Rationale
-hwaccel cuda -hwaccel_output_format cuda	Instructs FFmpeg to utilize the NVDEC hardware decoder and keep the decoded, uncompressed frames natively in the GPU's

	VRAM surface.[48] This prevents the frames from being downloaded to system RAM over the PCIe bus before being handed off to NVENC, fulfilling the strict zero-copy requirement.
-ss 00:01:12.050	By placing the start time *before* the input file flag (-i), FFmpeg parses the container metadata to jump rapidly to the nearest I-frame prior to the timestamp, decodes the stream invisibly up to the requested point, and begins writing the output file exactly at the 50ms offset.[39] Because the output is being actively re-encoded, there is no keyframe snapping on the final output.
-c:v av1_nvenc	Invokes the 9th-generation Blackwell AV1 hardware encoder, leveraging the dedicated ASIC to offload processing entirely from the CUDA cores and CPU.[47]
-preset p7 -tune hq	p7 enforces the absolute slowest, highest quality multi-pass NVENC profile available.[48] Crucially, -tune hq (High Quality) is selected rather than uhq (Ultra High Quality) to safely bypass the Blackwell artifacting bug, ensuring pristine frame rendering.[38]
-rc vbr -cq 19 -b:v 0	Shifts the encoder from generic Constant Bit Rate (CBR) to a visually lossless Constant Quality mode. A CQ value of 19 ensures visual indistinguishability from the source file while minimizing file size bloat. Setting -b:v 0 instructs the encoder to lift artificial bitrate ceilings and let the CQ algorithm strictly govern the compression.[38]
-bf 3	Forces the inclusion of 3 B-frames per GOP. This allows the encoder to reference frames both forwards and backwards in

	time, mathematically required to achieve maximum spatial compression efficiency in AV1.[38]

7. Synthesis of Concrete Engineering Solutions

To satisfy the operational constraints of the Intel Core Ultra 7 and the 16GB RTX 5080 Mobile inside a complex WSL2 Docker deployment, the architecture must undergo a series of immediate, low-level modifications.

First, the system must enforce strict IPC and memory boundaries. The Docker run sequence must include the --ipc=host flag to map the shared memory directly to the WSL2 virtual machine, completely overriding the 64MB Docker default that causes bus errors during tensor transfers.[2] Simultaneously, the Windows host .wslconfig file must be manually configured with a hard limit of memory=24GB to prevent runaway vmmem allocation from crashing the Windows 11 host during rapid process iteration and memory reclamation.[6]

Second, VRAM fragmentation must be actively mitigated. The pipeline must expose the environment variable PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True,max_split_size_mb:512 to the container environment. This explicitly instructs the PyTorch memory manager to utilize CUDA Virtual Memory Management to map discontiguous physical blocks into contiguous virtual space, ensuring that the 16GB limit is not exhausted artificially when alternating between Spoke models of drastically different tensor shapes (e.g., swapping ASR for Qwen).[16]

Third, the Hub-and-Spoke Locker pattern must be solidified and protected. The architecture must not attempt to migrate to NVIDIA Triton Inference Server, as its CUDA shared memory extensions are structurally incompatible with WSL2 network routing and WDDM paravirtualization, which will fail and result in massive, pipeline-halting PCIe traffic.[34] The custom Python "Locker" process must be retained, ensuring it allocates the shared buffer and remains alive indefinitely to prevent the underlying CUDA context from collapsing and invalidating the IPC handles passed to the Spoke models.[32] Furthermore, when passing PyNvVideoCodec frames into PyTorch, the pipeline must enforce OutputColorType.RGBP and utilize DLPack (torch.from_dlpack) to guarantee true zero-copy transfers.[36]

Finally, stream copy snapping must be eliminated. The -c copy methodology for the 50-millisecond perceptual cuts must be abandoned, as GOP I-frame mapping structurally prohibits millisecond accuracy without decoding.[37] The pipeline must implement the provided Blackwell AV1 NVENC command string to execute a fully hardware-accelerated re-encode of the target segment. This ensures perfect frame extraction, keeps the memory strictly on the GPU via -hwaccel_output_format cuda, and steers clear of the documented Blackwell uhq 10-bit encoding anomaly.[48]

By systematically tuning the PyTorch CUDA Caching Allocator, securely tying CUDA IPC handles to a persistent host process, and migrating to native Blackwell AV1 hardware re-encoding, the pipeline can be completely insulated against memory fragmentation, dangling pointer faults, and perceptual desynchronization. The integration of these exhaustive engineering implementations will yield a highly resilient, frame-accurate AI editing pipeline capable of operating flawlessly within strict mobile edge hardware thresholds.

Works cited

1. How can the shared memory size (/dev/shm) be incre... - Databricks Community - 12207, accessed March 7, 2026, https://community.databricks.com/t5/data-engineering/how-can-the-shared-memory-size-dev-shm-be-increased-on/td-p/12207
2. Device out of space error when torch.save with multiprocessing in docker - PyTorch Forums, accessed March 7, 2026, https://discuss.pytorch.org/t/device-out-of-space-error-when-torch-save-with-multiprocessing-in-docker/167869
3. Training fails due to memory exhaustion when running in a python multiprocessing.Process, accessed March 7, 2026, https://discuss.pytorch.org/t/training-fails-due-to-memory-exhaustion-when-running-in-a-python-multiprocessing-process/202773
4. Docker shared memory issue and solution · Issue #369 · RobotLocomotion/spartan - GitHub, accessed March 7, 2026, https://github.com/RobotLocomotion/spartan/issues/369
5. Training crashes due to - Insufficient shared memory (shm) - nn.DataParallel - vision, accessed March 7, 2026, https://discuss.pytorch.org/t/training-crashes-due-to-insufficient-shared-memory-shm-nn-dataparallel/26396
6. Docker desktop eats all memory and crashes - Stack Overflow, accessed March 7, 2026, https://stackoverflow.com/questions/64612117/docker-desktop-eats-all-memory-and-crashes
7. WSL2 backend has a potential memory leak · Issue #13022 · docker/for-win - GitHub, accessed March 7, 2026, https://github.com/docker/for-win/issues/13022
8. Docker Desktop eats RAM past WSL limit set in .wslconfig file, accessed March 7, 2026, https://forums.docker.com/t/docker-desktop-eats-ram-past-wsl-limit-set-in-wslconfig-file/128150
9. When using torch.multiprocessing for CUDA IPC, memory cannot be released. #171039, accessed March 7, 2026, https://github.com/pytorch/pytorch/issues/171039
10. Memory allocation to docker containers after moving to WSL 2 in Windows - Stack Overflow, accessed March 7, 2026, https://stackoverflow.com/questions/62405765/memory-allocation-to-docker-containers-after-moving-to-wsl-2-in-windows

11. Limiting Docker Memory Usage on Windows - DEV Community, accessed March 7, 2026, https://dev.to/sarahcssiqueira/limiting-docker-memory-usage-on-windows-5c45
12. Guide to run CUDA + WSL + Docker with latest versions (21382 Windows build + 470.14 Nvidia), accessed March 7, 2026, https://forums.developer.nvidia.com/t/guide-to-run-cuda-wsl-docker-with-latest-versions-21382-windows-build-470-14-nvidia/178365?page=2
13. Platform: Windows 10 using WSL2 w/GPU - fastai - fast.ai Course Forums, accessed March 7, 2026, https://forums.fast.ai/t/platform-windows-10-using-wsl2-w-gpu/73521
14. Pinned Memory Allocation Fails Above 500 MB in Docker on WSL2 · Issue #14078 · microsoft/WSL - GitHub, accessed March 7, 2026, https://github.com/microsoft/WSL/issues/14078
15. A guide to PyTorch's CUDA Caching Allocator | Zach's Blog, accessed March 7, 2026, https://zdevito.github.io/2022/08/04/cuda-caching-allocator.html
16. Vram issues with low input count · Issue #1242 · graphdeco-inria/gaussian-splatting - GitHub, accessed March 7, 2026, https://github.com/graphdeco-inria/gaussian-splatting/issues/1242
17. Boosting GPU Memory Efficiency with PyTorch and RAPIDS RMM | by Ion - Medium, accessed March 7, 2026, https://medium.com/@ion.stefanache0/boosting-gpu-memory-efficiency-with-pytorch-and-rapids-rmm-a0b1a02186e3
18. Has anyone determined how to actually "set max_split_size_mb to avoid fragmentation" as the CUDA error suggests? : r/StableDiffusion - Reddit, accessed March 7, 2026, https://www.reddit.com/r/StableDiffusion/comments/15pg318/has_anyone_determined_how_to_actually_set_max/
19. CUDA semantics — PyTorch 2.10 documentation, accessed March 7, 2026, https://docs.pytorch.org/docs/stable/notes/cuda.html
20. cudaMallocAsync cause too much fragmentation. · Issue #110194 - GitHub, accessed March 7, 2026, https://github.com/pytorch/pytorch/issues/110194
21. [Bug] torch.OutOfMemoryError: CUDA out of memory for 16GB Vram while trying to inference gemma-3-12b-IT · Issue #5576 · sgl-project/sglang - GitHub, accessed March 7, 2026, https://github.com/sgl-project/sglang/issues/5576
22. [Issue]: [gfx1151] pytorch is allocating vram but then loads in shared GPU memory instead, causing OOM · Issue #3032 · ROCm/TheRock - GitHub, accessed March 7, 2026, https://github.com/ROCm/TheRock/issues/3032
23. how to limit vram usage? : r/StableDiffusion - Reddit, accessed March 7, 2026, https://www.reddit.com/r/StableDiffusion/comments/17h4zd7/how_to_limit_vram_usage/
24. RuntimeError: CUDA out of memory. How can I set max_split_size_mb? - Stack Overflow, accessed March 7, 2026, https://stackoverflow.com/questions/73747731/runtimeerror-cuda-out-of-memory-how-can-i-set-max-split-size-mb
25. CUDA out of memory issue #1654 - GitHub, accessed March 7, 2026,

https://github.com/ultralytics/ultralytics/issues/1654
26. 4.15. Interprocess Communication — CUDA Programming Guide, accessed March 7, 2026, https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/inter-process-communication.html
27. Synchronization between CUDA applications - Stack Overflow, accessed March 7, 2026, https://stackoverflow.com/questions/63561346/synchronization-between-cuda-applications
28. CUDA shared memory doesn't work (failed to open CUDA IPC handle: invalid device context) - DeepStream SDK - NVIDIA Developer Forums, accessed March 7, 2026, https://forums.developer.nvidia.com/t/cuda-shared-memory-doesnt-work-failed-to-open-cuda-ipc-handle-invalid-device-context/325917
29. Unexpected CUDA IPC handle behaviour - PyTorch Forums, accessed March 7, 2026, https://discuss.pytorch.org/t/unexpected-cuda-ipc-handle-behaviour/218867
30. CUDA error: an illegal memory access was encountered with reproduction, accessed March 7, 2026, https://discuss.pytorch.org/t/cuda-error-an-illegal-memory-access-was-encountered-with-reproduction/135532
31. PyTorch CUDA error: an illegal memory access was encountered - Stack Overflow, accessed March 7, 2026, https://stackoverflow.com/questions/68106457/pytorch-cuda-error-an-illegal-memory-access-was-encountered
32. Using CUDA IPC memory handles in pytorch, accessed March 7, 2026, https://discuss.pytorch.org/t/using-cuda-ipc-memory-handles-in-pytorch/17548
33. Shared-Memory Extension — NVIDIA Triton Inference Server, accessed March 7, 2026, https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/protocol/extension_shared_memory.html
34. CUDA IPC Handle failure in WSL2 forces CPU memory fallback, causing massive PCIe bottlenecks in BLS pipelines #8670 - GitHub, accessed March 7, 2026, https://github.com/triton-inference-server/server/issues/8670
35. PyNvVideoCodec - Get Started - NVIDIA Developer, accessed March 7, 2026, https://developer.nvidia.com/pynvvideocodec
36. PyNvVideoCodec API Programming Guide - NVIDIA Docs, accessed March 7, 2026, https://docs.nvidia.com/video-technologies/pynvvideocodec/pynvc-api-prog-guide/index.html
37. Frame-accurate video cuts — can FFmpeg help without increasing file size? - Reddit, accessed March 7, 2026, https://www.reddit.com/r/ffmpeg/comments/1nw03if/frameaccurate_video_cuts_can_ffmpeg_help_without/
38. How can I use CRF encoding with nvenc in ffmpeg? - Super User, accessed March

7, 2026, https://superuser.com/questions/1236275/how-can-i-use-crf-encoding-with-nvenc-in-ffmpeg
39. How to cut at exact frames using ffmpeg? - Super User, accessed March 7, 2026, https://superuser.com/questions/459313/how-to-cut-at-exact-frames-using-ffmpeg
40. How to cut a video with FFmpeg with no (or minimal) re-encoding? - Super User, accessed March 7, 2026, https://superuser.com/questions/1850814/how-to-cut-a-video-with-ffmpeg-with-no-or-minimal-re-encoding
41. Using ffmpeg to cut videos with more precision than key frames allow, accessed March 7, 2026, https://video.stackexchange.com/questions/16750/using-ffmpeg-to-cut-videos-with-more-precision-than-key-frames-allow
42. How would I cut video in ffmpeg, while only re-encoding the very first and last frames?, accessed March 7, 2026, https://www.reddit.com/r/ffmpeg/comments/xaoybr/how_would_i_cut_video_in_ffmpeg_while_only/
43. Add AV1 NVENC Ultra High Quality Mode (RTX 5080)/ svt-av1 Hybrid · Issue #733 - GitHub, accessed March 7, 2026, https://github.com/rigaya/NVEnc/issues/733
44. Nvidia 50-series AV1 + HEVC improvements - Reddit, accessed March 7, 2026, https://www.reddit.com/r/AV1/comments/1hvmk0n/nvidia_50series_av1_hevc_improvements/
45. Does Nvidia 5080 Produce BETTER AV1 files than Nvidia 4080? - Reddit, accessed March 7, 2026, https://www.reddit.com/r/AV1/comments/1igtppi/does_nvidia_5080_produce_better_av1_files_than/
46. NVIDIA NVENC Obs Guide | GeForce News, accessed March 7, 2026, https://www.nvidia.com/en-us/geforce/guides/broadcasting-guide/
47. Improving Video Quality and Performance with AV1 and NVIDIA Ada Lovelace Architecture, accessed March 7, 2026, https://developer.nvidia.com/blog/improving-video-quality-and-performance-with-av1-and-nvidia-ada-lovelace-architecture/
48. Using FFmpeg with NVIDIA GPU Hardware Acceleration, accessed March 7, 2026, https://docs.nvidia.com/video-technologies/video-codec-sdk/13.0/ffmpeg-with-nvidia-gpu/index.html
49. HWAccelIntro – FFmpeg, accessed March 7, 2026, https://trac.ffmpeg.org/wiki/HWAccelIntro
50. NVENC HEVC UHQ + highbitdept leads to artfiacts in ffmpeg with Blackwell, accessed March 7, 2026, https://forums.developer.nvidia.com/t/nvenc-hevc-uhq-highbitdept-leads-to-artfiacts-in-ffmpeg-with-blackwell/354084
51. How to get FFMPEG to use more GPU when encoding - Stack Overflow, accessed March 7, 2026, https://stackoverflow.com/questions/63285578/how-to-get-ffmpeg-to-use-more

-gpu-when-encoding
52. AV1 NVENC Config for FFMPEG - GitHub Gist, accessed March 7, 2026, https://gist.github.com/Nicklas373/6cf92b3594561bff326697a93ffb2e5b